

Digital Image Processing

Homework 2

{Low-Luminosity Enhancement + Sharpness Enhancement + Denoise}

I. Low-Luminosity Enhancement

(1) Adjusting Luminosity:

Luminosity is calculated as a weighted sum of the red, green, and blue values: $0.21 \times \text{Red} + 0.72 \times \text{Green} + 0.07 \times \text{Blue}$. This formula mimics human color perception by emphasizing the green channel. The function enhances pixels only if their luminosity falls below a **specified threshold (100 in this case)**, which represents low-luminosity areas. For these darker pixels, the function proportionally increases the RGB values, resulting in a balanced enhancement that preserves the image's natural appearance and prevents overexposure in already bright areas.

(2) Low-Luminosity Enhancement:

This 'Enhancement' function reads and writes BMP files, loading pixel data into memory and applying the 'Luminosity' function to adjust individual pixels based on their calculated luminosity. By omitting adjustments for higher-luminosity pixels, the function selectively enhances only the darker areas, thereby improving visibility without altering the overall brightness of the image.

(3) Main Program Execution:

The 'main' function iterates over a list of input BMP files, while the **'luminosityboost' parameter (50 in this case)** controls the level of enhancement, allowing for customization to optimize the effect for specific images. This flexibility facilitates tailored enhancements based on the lighting conditions present in each image.

Summary:

This code is advantageous for images captured in low-light conditions or shadows. Such selective enhancement can improve details in night photography or underexposed areas, making it ideal for applications in digital photography, medical imaging, and surveillance, where visual clarity in darker regions is essential.

II. Sharpness Enhancement

(1) Using Convolution Kernel:

The essence of the sharpening process is rooted in the 'Kernel' function. This function utilizes a convolution kernel, which is a matrix of integers that dictates how the values of surrounding pixels are combined to enhance sharpness. For each pixel, the function calculates the weighted sum of neighboring pixels based on the kernel values and constrains the resulting values to remain within the standard range of 0 to 255, thereby preventing overflow artifacts.

(2) Sharpening Process:

The 'Sharpness' function reads the pixel data of an image into a two-dimensional vector, applies a 'Kernel' function to each pixel, and then writes the transformed pixel data back to an output BMP file.

(3) Define Two Sharpening Kernels:

Strong Sharpening Kernel: A simple matrix that emphasizes edge contrast by increasing the intensity of the center pixel while diminishing that of its surrounding pixels.

Unsharp Masking Kernel: A larger kernel with a greater central weight that produces a more refined sharpening effect, effectively balancing edge enhancement with noise reduction.

Summary:

This code is designed to improve image detail by making edges and textures more distinct, which is particularly useful in fields like medical imaging, photography, and pattern recognition. It employs customizable kernels that allow users to adjust the intensity and style of sharpening to fit specific needs, providing flexibility in how details are enhanced. Additionally, the code includes careful handling of BMP-specific padding, ensuring that each row remains properly aligned.

III. Denoise

(1) Apply Median Filter:

The `'MedianFilter'` function operates by examining the neighborhood of a pixel, which is defined by the `'kernelSize'`. It collects the RGB values from this neighborhood, sorts them, and replaces the central pixel's value with the median of the collected values. This method effectively eliminates "salt-and-pepper" noise by smoothing out extreme values. While the filter is proficient at reducing noise by substituting outliers with median values within a local window, it can blur fine details and textures, particularly when larger kernel sizes are used. Consequently, it is well-suited for low-complexity images but may not perform as effectively on natural or high-detail images.

(2) Apply Bilateral Filter:

The `'BilateralFilter'` function combines spatial proximity and color similarity to smooth images while preserving edges. Each neighboring pixel contributes to the final value based on weights derived from spatial distance (controlled by `'sigmaSpatial'`) and color difference (controlled by `'sigmaColor'`). This method effectively reduces noise in homogeneous regions while maintaining edges, as it assigns higher weights to pixels with similar colors, thereby preserving important details. By adjusting `'sigmaSpatial'` and `'sigmaColor'`, the filter can be fine-tuned to balance the intensity of smoothing with edge retention, making it particularly suitable for natural images where edge clarity is crucial. However, this filter is computationally intensive due to the added complexity of calculating color-based weights.

(3) Main Program Execution:

The `'main'` function establishes filter parameters and applies the corresponding filters to images with varying noise characteristics. It is also beneficial that the function can evaluate the effectiveness of each filter using a variety of noise images.

(4) Evaluate Denoising Results:

I utilized the **Structural Similarity Index (SSIM)** to assess the denoising outcomes. Since SSIM is typically applied to grayscale images, I convert any color images (i.e.,

those with three channels) to grayscale. I then printed the data type and value range to confirm the conversion and the new range. Next, I calculate the SSIM score between the two grayscale images, setting the `data\_range` parameter to specify the intensity range. This ensures an accurate SSIM calculation based on the distribution of pixel values. Finally, I printed the resulting SSIM score.

SSIM score between the denoised image and the original image is as follows:

Noisy Image	Filter	Parameters	Denoised Image	SSIM
input3	Median	kernelSize = 3	output3_1	0.94
input3	Median	kernelSize = 5	output3_2	0.89
input4	Bilateral	kernelSize = 9	output4_1	0.75
		sigmaSpatial = 10		
		sigmaColor = 100		
input4	Bilateral	kernelSize = 7	output4_2	0.68
		sigmaSpatial = 8		
		sigmaColor = 60		

Performance Analysis:

Median Filter

For input3 with kernelSize = 3: The SSIM score is 0.94, indicating high structural similarity between the denoised and original image, meaning effective noise reduction with minimal detail loss.

For input3 with kernelSize = 5: The SSIM score drops slightly to 0.89, suggesting that increasing the kernel size may result in a minor loss of detail, although it still maintains a good level of similarity.

Bilateral Filter

For input4 with sigmaSpatial = 10 and sigmaColor = 100: The SSIM score is 0.75, showing moderate similarity. While this filter preserves edges, the high `sigmaColor` value may have impacted fine details.

For input4 with sigmaSpatial = 8 and sigmaColor = 60: The SSIM score decreases further to 0.68, indicating that these parameter settings might lead to more detail loss or less effective noise removal.